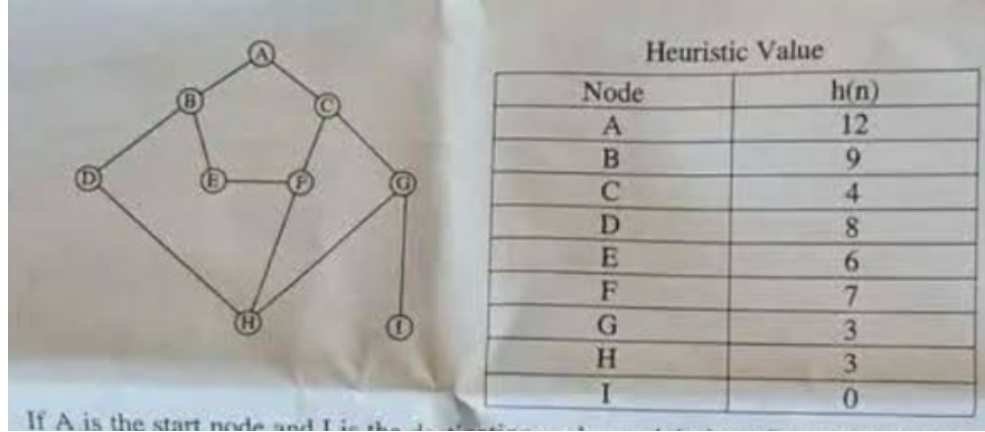


**Previous year Question
BCS and Others**

46th BCS
(CSE)

- What is Artificial Neural Network (ANN)
- Why does a Heuristic function always produce a positive value? Explain
- Consider the following tree and heuristic value $h(n)$ at each node of the tree. if A is the start node and I is the destination node, explain how the best first search algorithm can be used to find the destination node. Also show the path to the destination.



46th BCS
(ICT)

- Differentiate between supervised and unsupervised learning.
- Present the KNN algorithm. What is the significance of the value of K in KNN algorithm?
- What are the limitations of k-means clustering? How can you minimize these limitations?

47th BCS
(CSE)

- Differentiate between Supervised learning and Unsupervised learning with examples
- How do neural networks distinguish useful features from non-useful features in Computer Vision?
- What is Explainable AI (XAI)? How does it work? Explain with examples.

47th BCS
(ICT)

- Explain the difference between Deterministic Finite Automata (DFA) and Non Deterministic Finite Automata (NFA).
- Prove that for every NFA, there exists an equivalent DFA that recognizes the same language.

50th BCS
(CSE)

- ব্যাখ্যা করুন- কেন First Order Logic (FOL) কে generalization of propositional logic (PL) বলা হয়?
- নিচের বাক্যগুলো First Order Logic এ লিখুন-
 - You can always fool some of the people
 - All purple mushrooms are poisonous
- Heuristic Function কী? এটি কিভাবে search performance কে উন্নত করে? ব্যাখ্যা করুন।
- AI -এ probabilistic reasoning এর জন্য Bayesian Network এর প্রয়োজন ব্যাখ্যা করুন। এটি deterministic rule-based system o থেকে কীভাবে আলাদা?
- Confusion matrix কী এবং এটি কেন ব্যবহার হয়?

Artificial Intelligence (AI)

Artificial Intelligence (AI) refers to the simulation of human intelligence in machines which helps in allowing them to think and act like humans. It involves creating algorithms and systems that can perform tasks which require human abilities such as visual perception, speech recognition, decision-making and language translation.

History -

1. **John McCarthy** is the person most widely recognized as the **Father of Artificial Intelligence (AI)** as he first proposed the term Artificial Intelligence in 1965.
2. **Arthur Samuel** is the person recognized as the **Father of Machine Learning (ML)** as he first proposed the term in 1959.
3. **Geoffrey Hinton** is called the **Godfather of Deep Learning** for his foundational work on neural networks.
4. **Yann LeCun** – A pioneer of Convolutional Neural Networks (CNNs).
5. **Richard Feynman** most commonly called the **Father of Quantum Computing** as he proposed Quantum Computer in **1981**. **David Deutsch** developed the theoretical model of the universal quantum computer in **1985**.

Following are some **key characteristics** of Artificial Intelligence (AI) -

1. Learning Ability - AI systems can learn from data and improve performance over time
2. Reasoning and Decision-Making - AI can analyze information and make logical decisions
3. Problem-Solving Capability - AI can solve complex problems by breaking them into smaller tasks
4. Knowledge Representation - AI systems store and organize knowledge in structured forms
5. Natural Language Processing (NLP) - AI can understand, interpret, and generate human language
6. Perception - AI can interpret sensory data such as images, audio, and video.
7. Automation - AI enables machines to perform tasks automatically with minimal human intervention
8. Adaptability - AI systems can adjust to new inputs and environments.
9. Pattern Recognition - AI can identify patterns in large datasets
10. Autonomy - AI systems can operate independently without constant human control.

Advantages of AI -	Disadvantages of AI -
● Automation of Tasks ● High Accuracy ● Reduced Errors ● 24/7 Availability ● Handling Dangerous Tasks ● Fast Decision-Making ● Data Analysis and Insights ● Improved User Experience	● High Cost of Development ● Job Displacement ● Lack of Creativity and Emotion ● Dependence on Data ● Security and Privacy Risks ● Limited Understanding ● Ethical Issues

	How AI and ML can be used to provide more secure and professional cyber-security in organizations that can be very strict in data integrity and security like banking sector
--	---

AI and ML can significantly improve cybersecurity in banks, but the safest approach is **AI-assisted security rather than blindly giving AI control**. In a banking environment, confidentiality, integrity, availability, auditability, and regulatory compliance must come first.

Where AI/ML can help

1. **Fraud and anomaly detection** - Learn normal transaction behavior for customers and accounts. Detects unusual transfers, login locations, transaction amounts, device behavior, or transaction timing. Assign a risk score and trigger additional verification rather than automatically blocking everything.

2. **Real-time cyberattack detection** - ML models can analyze network traffic, authentication logs, endpoint activity, and application logs. They can identify patterns associated with ransomware, credential theft, insider threats, DDoS attacks, and account takeover. This can complement traditional signature/rule-based security systems.
3. **User and Entity Behavior Analytics (UEBA)** - Establish a behavioral baseline for employees, applications, servers, and privileged accounts. For example, if an employee who normally accesses five internal systems suddenly downloads a large volume of sensitive data at 3 AM, the system can flag it for investigation.
4. **AI-assisted Security Operations Center (SOC)** - AI can correlate millions of security events and prioritize the most dangerous ones. Instead of analysts investigating thousands of alerts, AI can produce something like:
High probability of compromised privileged account — unusual login + abnormal database access + large data transfer.
 Analysts can then verify the finding before taking action.
5. **Phishing and social-engineering detection** - ML/NLP can analyze emails, URLs, attachments, sender behavior, and message characteristics. AI can identify sophisticated phishing attempts that traditional filters might miss.
6. **Vulnerability management** - AI can help prioritize vulnerabilities based on exploitability, asset importance, exposure, and business impact. This is better than simply saying "patch everything with a critical CVSS score first."
7. **Identity and access security** - AI can continuously evaluate authentication risk. A login from a recognized device might be considered low risk, while a login combined with impossible travel, unusual device characteristics, and abnormal behavior could require MFA or additional verification.

For example, if an ML model detects a suspicious transaction, it should produce:

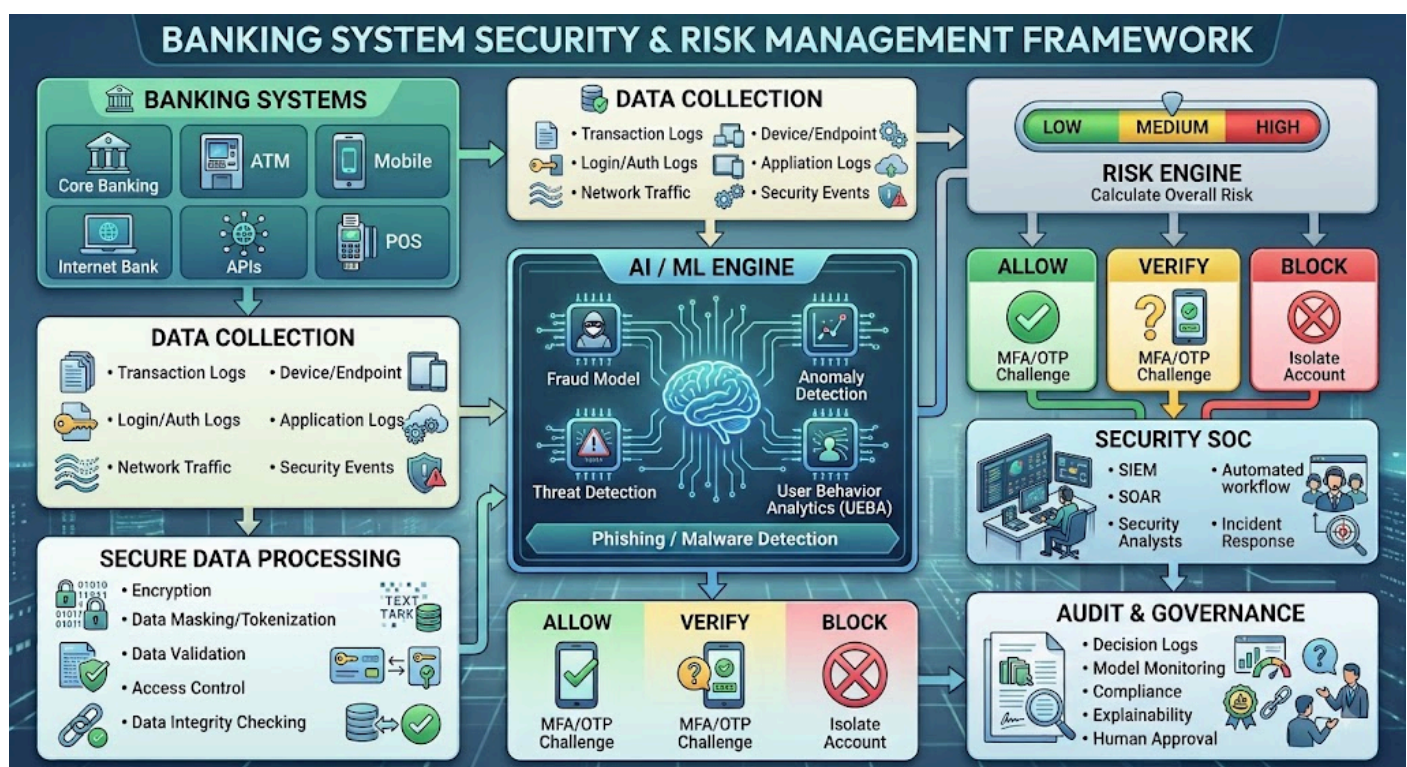
Transaction ID: 874291

Risk Score: 0.97

Reason:

- Unusual amount
- New beneficiary
- Unusual device
- Abnormal geographic pattern
- Behavioral deviation

Recommended action: Step-up authentication

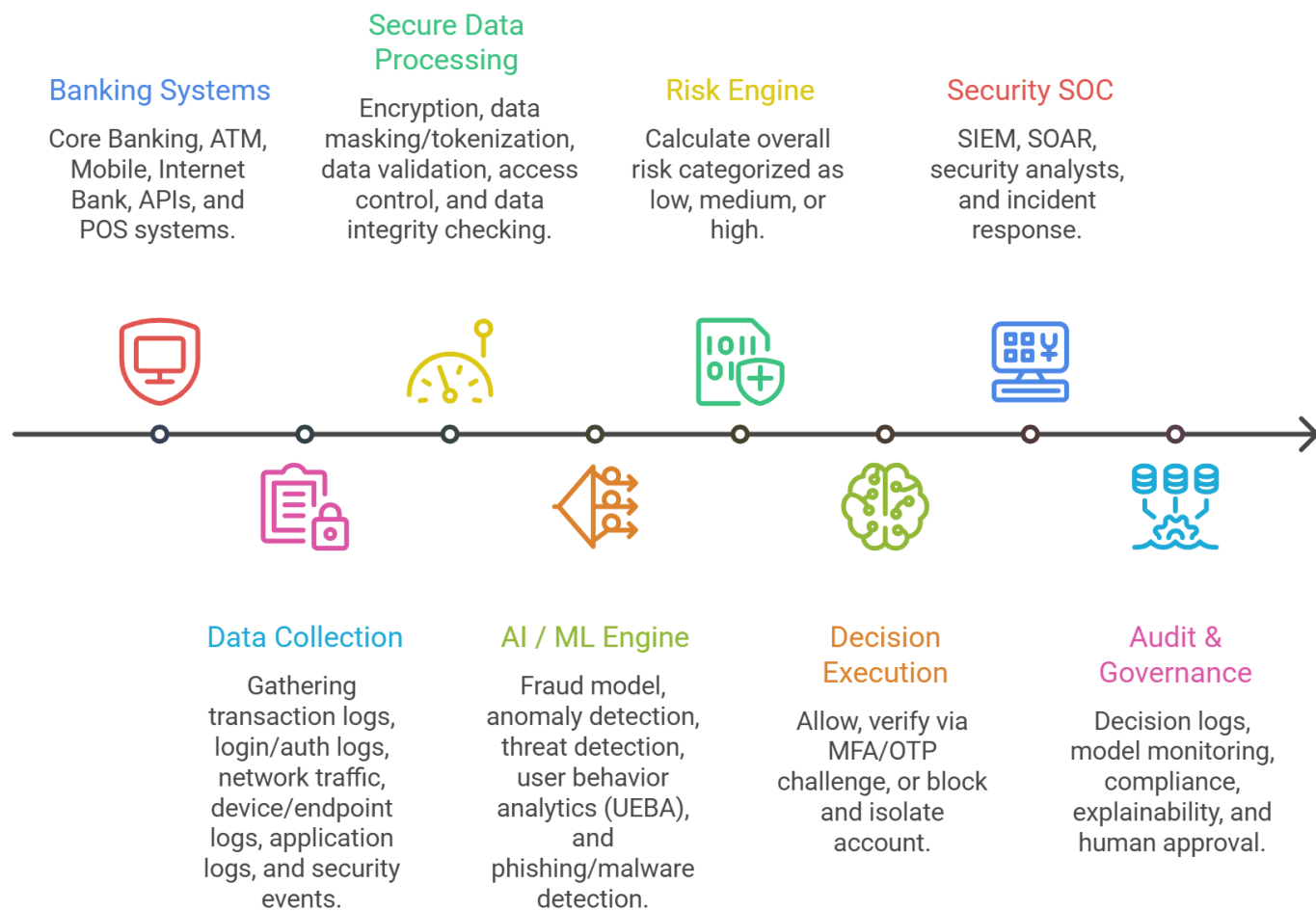


Defense against AI-specific attacks - Banks also need to protect ML systems from attacks such as:

- Data poisoning — attackers manipulate training data.
- Adversarial inputs — carefully crafted inputs fool a model.
- Model theft — attackers attempt to reproduce a proprietary model.
- Prompt injection — relevant when an LLM is connected to internal systems.
- Data leakage — sensitive information is accidentally exposed through an AI system.
- Model drift — legitimate behavior changes and the model becomes inaccurate.
- False positives — legitimate customers are repeatedly blocked.
- False negatives — sophisticated attacks are missed.

Therefore, an AI security system needs its own security monitoring and governance.

Banking Security Operations Workflow



Agents of AI

An artificial intelligence (AI) agent is a software program that can interact with its environment, collect data and use that data to perform self-directed tasks that meet predetermined goals.

Humans set goals, but an AI agent independently chooses the best actions it needs to perform to achieve those goals. For example, consider a **customer support AI agent** that wants to resolve customer queries. The agent will automatically ask the customer different questions, look up information in internal documents and respond with a solution. Based on the customer responses, it determines if it can resolve the query itself or pass it on to a human.

Characteristics	Components of an AI Agent
a. Autonomous b. Goal driven	1. Sensors/Input: Collect information from the environment. 2. Knowledge Base: Stores facts, rules, or learned information.

c. Perceptive d. Adaptable e. Collaborative f. Rational	3. Reasoning Engine: Analyzes information and makes decisions. 4. Planning Module: Determines the sequence of actions. 5. Learning Module: Improves performance using experience. 6. Actuators/Output: Executes actions or provides responses.
--	---

How do AI Agents Work?

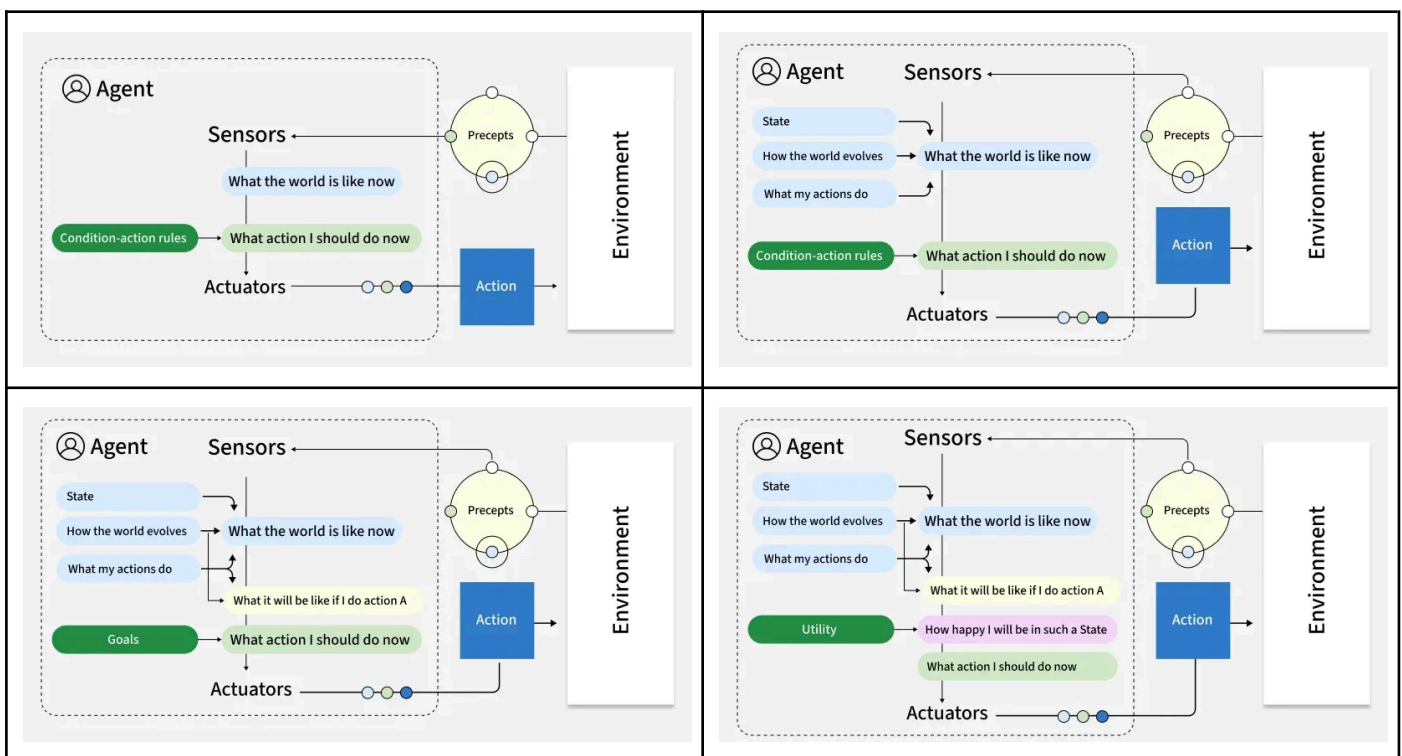
Input (Perception) -> Understanding the request -> Reasoning and Planning -> Decision Making -> Action Execution -> Learning and Improvement

Suppose you ask an AI assistant: "Find the cheapest flight from Dhaka to Singapore next Friday."

The AI agent:

1. Understand your request.
2. Identifies the departure city, destination, and travel date.
3. Searches airline and travel websites.
4. Compares flight prices.
5. Select the best option.
6. Presents the results or books the ticket (if authorized).

Different types of agents of AI				
Agent Type	Description	Working Procedure	Key Characteristics	Example
Simple Reflex Agent	Acts only on the current percept using predefined rules.	Receives the current input → matches it with a condition-action rule → performs the corresponding action.	No memory, no learning, rule-based, suitable for simple environments.	Automatic door, thermostat
Model Based Reflex Agent	Maintains an internal model of the environment to make decisions.	Receives input → updates its internal model using current and past information → selects an action based on the model.	Uses memory, handles partially observable environments, more intelligent than simple reflex agents.	Robot vacuum that remembers obstacles
Goal Based Agent	Chooses actions that help achieve a specific goal.	Analyzes the current state → compares it with the desired goal → plans and selects actions that lead to the goal.	Goal-oriented, planning capability, evaluates multiple possible actions before acting.	GPS navigation system
Utility Based Agent	Selects the action that provides the highest utility (best outcome).	Evaluates all possible actions using a utility function → calculates the expected utility → chooses the action with the highest utility.	Optimizes performance, considers trade-offs, handles uncertainty effectively.	Self-driving car choosing the safest and fastest route
Learning Agent	Learns from experience and improves its performance over time.	Observes the environment → takes action → receives feedback → updates its knowledge or model → improves future decisions.	Adaptive, self-improving, learns from data and experience, suitable for dynamic environments.	Spam email filter, recommendation system



Overfitting and Underfitting are two common problems in machine learning where a model fails to generalize properly to new, unseen data.

Overfitting occurs when a model learns the training data too well, including its noise and irrelevant patterns. As a result, it performs very well on training data but poorly on new data. Example: A student memorizes all questions from a textbook but cannot solve slightly different questions in an exam.

Causes of overfitting -

1. Model is too complex
2. Too many features
3. Training data is too small
4. Noisy or irrelevant training data
5. Training for too many epochs
6. Insufficient regularization

Prevention -

1. Use more training data
2. Simplify the model
3. Remove irrelevant features
4. Use regularization such as L1/L2
5. Use dropout in neural networks
6. Apply cross-validation
7. Use early stopping
8. Use data augmentation where appropriate

Underfitting occurs when a model is too simple to learn the important patterns in the training data. Therefore, it performs poorly on both training and unseen data. Example: A student studies only a few basic concepts and cannot answer either basic or advanced questions properly.

Causes -

1. Model is too simple
2. Too few features
3. Insufficient training
4. Excessive regularization
5. Poor feature selection
6. Training data does not adequately represent the problem

Prevention -

1. Use a more complex model
2. Add relevant features
3. Train the model for longer
4. Reduce excessive regularization
5. Improve feature engineering
6. Use an appropriate machine-learning algorithm

Different types of learning of AI

Supervised

Supervised learning is a type of machine learning in which the model is trained using **labeled data**. Each training example contains an input and its correct output (label). The goal is to learn the relationship between inputs and outputs so that the model can accurately predict the output for new, unseen data.

	Examples: 1. Email spam detection 2. Image recognition 3. House price prediction 4. Disease diagnosis Characteristics - 1. Uses labeled data 2. Requires human supervision 3. Predictive capability 4. Used for classification and regression 5. High accuracy with quality data 6. Error-based learning
Unsupervised	Unsupervised learning is a type of machine learning in which the model is trained using unlabeled data. The model identifies hidden patterns, structures, or relationships in the data without knowing the correct outputs. Examples: 1. Customer segmentation 2. Market basket analysis 3. Document clustering 4. Anomaly detection Characteristics: 1. Uses unlabeled data 2. Finds hidden patterns 3. Performs clustering and grouping 4. Requires less human intervention 5. Useful for data exploration 6. Supports dimensionality reduction
Reinforcement	Reinforcement learning is a type of machine learning in which an agent learns by interacting with an environment. The agent receives rewards for correct actions and penalties for incorrect actions. Its objective is to maximize the total reward over time. Examples: 1. Self-driving cars 2. Robotics 3. Game playing AI (Chess, Go, Atari) 4. Autonomous drones 5. Traffic signal optimization Characteristics: 1. Learns through trial and error 2. Uses rewards and penalties 3. Goal-oriented learning 4. Requires continuous interaction 5. Adapts to changing environments 6. Suitable for sequential decision-making

Comparison among different types of learning algorithms of AI

Criterion	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Input Data	Labeled Data	Unlabeled Data	No Predefined Data Set (Data is generated dynamically through interaction).
Feedback	Direct	No External Feedback (The model works on its own to find patterns).	Reward/Penalty Signal (Feedback is a scalar value received after an action, often delayed).
Main Goal	Prediction or Classification (Map input to a known output/label).	Pattern Discovery (Find hidden structures, relationships, or groupings).	Maximize Cumulative Reward (Learn an optimal sequence of actions, or "policy").

Comparison among different types of learning algorithms of AI			
Criterion	Supervised Learning	Unsupervised Learning	Reinforcement Learning
Typical Problems	Classification Regression	Clustering Dimensionality reduction	Sequential decision making (Game Playing, Robotics control).
Core Process	Error Minimization between the model's prediction and the true label.	Feature Extraction and grouping based on similarity or difference.	Trial and Error approach driven by the State-Action-Reward cycle.
Learning Environment	Static/Offline (Learning is performed on a fixed training data set).	Static/Offline (Learning is performed on a fixed training data set).	Dynamic/Interactive (Learning happens in real-time as the agent interacts with its environment).
Computational Load	Generally Less Complex than RL, focusing on data fitting and prediction.	Can be Complex depending on the technique (e.g., finding optimal clusters).	Highly Complex, requiring simulation, exploration, and long-term planning.
Key Algorithms	- Linear Regression - Logistic Regression - Decision Trees - Support Vector Machines (SVM) - k-Nearest Neighbours (kNN) - Naïve Bayes - Random Forests	- K-Means Clustering, - Principal Component Analysis (PCA), - Hierarchical Clustering.	- Q-Learning - Deep Q-Networks (DQN) - Markov decision processes (MDPs) - Bellman equation

	Discuss K-Nearest Neighbors (KNN) algorithm with an example
--	---

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression tasks. It works on the principle that similar data points are likely to have similar outputs. KNN classifies a new data point based on the classes of its nearest neighboring data points.

Working Procedure of KNN Algorithm

- Select the value of K - Choose the number of nearest neighbors (K) to consider for prediction.
- Calculate Distance - Calculate the distance between the new data point and all training data points.
Common distance measures include:
 - Euclidean distance
 - Manhattan distance
 - Minkowski distance
- Find the K Nearest Neighbors - Arrange the data points according to their distance from the new point. Select the closest K data points.
- Make Prediction

- a. For classification: The class with the highest number of neighbors is assigned to the new data point.
 - b. For regression: The average value of the K nearest neighbors is used as the prediction.
5. Output the Result - The algorithm returns the predicted class or value.

Fruit	Weight (grams)	Color Value
Apple	150	7
Apple	160	8
Apple	170	9
Orange	120	3
Orange	130	4
Orange	140	5

Example of K-Nearest Neighbors (KNN) Algorithm - Suppose we want to classify whether a fruit is an Apple or an Orange based on its weight and color. Now, a new fruit is given: Weight = 155 grams and Color Value = 7. We need to determine whether it is an Apple or an Orange.

Step 1: Choose the Value of K - Assume K = 3. This means the algorithm will consider the 3 nearest fruits to classify the new fruit.

Step 2: KNN calculates the distance between the new fruit and every fruit in the training dataset using **Euclidean Distance**:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_1 - y_2)^2}$$

Apple (150, 7)	$\sqrt{(150-155)^2 + (7-7)^2} = 5$	Very close
Apple (160, 8)	$\sqrt{(160-155)^2 + (8-7)^2} = 5$	Close
Apple (170, 9)	$\sqrt{(140-155)^2 + (7-7)^2} = 5$	Close
Orange (120, 3)	$\sqrt{(120-155)^2 + (7-7)^2} = 5$	Far away
Orange (130, 4)	$\sqrt{(130-155)^2 + (7-7)^2} = 5$	Far away
Orange (140, 5)	$\sqrt{(140-155)^2 + (7-7)^2} = 5$	Far away

Step 3: Select the K Nearest Neighbors - The 3 closest neighbors are:

Apple (150, 7), Apple (160, 8) and Apple (170, 9)

Step 4: Perform Classification - Among the 3 nearest neighbors - Apple = 3 votes, Orange = 0 votes. Therefore, the majority class is Apple.

Step 5: Prediction - The new fruit is classified as: Apple

Discuss how the k-means clustering algorithm works with an example. What are the limitations of k-means clustering? How can you minimize these limitations?

K-Means clustering is an **unsupervised machine learning algorithm** used to divide data into **K distinct groups (clusters)** based on similarity. It groups data points so that points within the same cluster are more similar to each other than to points in other clusters.

Example - A company wants to group customers into different categories based on their annual income and spending score. The goal is to identify customer groups for targeted marketing.

The company wants to divide customers into 2 groups (which means k = 2).

Customer	Annual Income (thousand)	Spending Score
A	15	20
B	18	25
C	20	30
D	60	80
E	65	85
F	70	90

Step 1: Select Initial Centroids - The algorithm randomly selects two initial cluster centers (centroids). For example:

- a. Centroid 1 = Customer A (15, 20)
- b. Centroid 2 = Customer D (60, 80)

Step 2: Calculate Distance and Assign Clusters - K-Means calculates the distance between each customer and both centroids using Euclidean distance:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_1 - y_2)^2}$$

Each customer is assigned to the nearest centroid. Cluster Assignment:

Cluster 1 (Low Income, Low Spending): Customer A, Customer B, Customer C

Cluster 2 (High Income, High Spending): Customer D, Customer E, Customer F

Step 3: Update Centroids - The new centroid is calculated by finding the average value of all points in each cluster.

New Centroid of Cluster 1:	Average Income = $(15+18+20)/3 = 17.67$ Average Spending Score = $(20+25+30)/3 = 25$	(17.67, 25)
New Centroid of Cluster 2:	Average Income = $(60+65+70)/3 = 65$ Average Spending Score = $(80+85+90)/3 = 85$	(65, 85)

Step 4: Repeat the Process - Recalculate distances using the new centroids. Reassign customers if necessary. Continue updating centroids until no customer changes its cluster. The clusters become stable.

Final Result - The customers are divided into two groups:

- Cluster 1: Low-Value Customers: Lower income + Lower spending score
Customers: A, B, C
- Cluster 2: High-Value Customers: Higher income + Higher spending score
Customers: D, E, F

Limitations of K-Means Clustering and their improvements

- Need to Specify the Value of K** - K-Means requires the number of clusters to be defined before training. Choosing an incorrect value of K can produce poor results. **Minimization:** Use methods such as - Elbow Method, Silhouette Score, Gap Statistics
- Sensitive to Initial Centroid Selection** - Randomly selected initial centroids can lead to different results. Poor initialization may produce incorrect clusters. **Minimization:** Use improved initialization techniques such as - K-Means++ algorithm and Run the algorithm multiple times and select the best result.
- Sensitive to Outliers and Noise** - Extreme values can significantly affect the centroid position. Outliers may create incorrect clusters. **Minimization:** Remove or handle outliers before clustering. Use robust clustering methods when necessary.
- Works Poorly with Non-Spherical Clusters** - K-Means assumes clusters are round and evenly distributed. It performs poorly when clusters have irregular shapes. **Minimization:** Use alternative algorithms such as - DBSCAN, Hierarchical clustering, Gaussian mixture models
- Sensitive to Feature Scaling** - Features with larger numerical ranges can dominate distance calculations. **Minimization:** Normalize or standardize data before applying K-Means.
- Requires Numeric Data** - K-Means relies on distance calculations, making it unsuitable for categorical data. **Minimization:** Convert categorical variables into numerical form using encoding techniques. Use algorithms designed for categorical data.

Deep Learning of AI

Deep learning (DL) is a specialized subset of Machine Learning (ML) that is the driving force behind many modern Artificial Intelligence (AI) systems. It is distinguished by its use of deep neural networks, multi-layered artificial neural networks, which are inspired by the structure and function of the human brain. **Deep Learning** is a subset of machine learning that uses **artificial neural networks with multiple layers** to learn complex patterns from large amounts of data.

Examples of Deep Learning Applications:

- Face recognition
- Voice assistants
- Self-driving cars
- Medical image analysis
- Language translation
- Generative AI systems

Characteristics of Deep Learning

- a. Uses Deep Neural Networks - Deep learning models consist of multiple layers of artificial neurons (input layer, hidden layers, and output layer) that enable complex pattern recognition.
- b. Automatic Feature Extraction - Unlike traditional machine learning, deep learning automatically learns important features from raw data without requiring manual feature engineering.
- c. Requires Large Amounts of Data - Deep learning models perform best when trained on large datasets such as images, videos, text, and audio.
- d. High Computational Requirements - Training deep learning models requires powerful hardware such as GPUs and TPUs due to complex calculations and large model sizes.
- e. Learn Hierarchical Representations - Lower layers learn simple features, while deeper layers learn more complex concepts. For example, image models learn edges first, then shapes, and finally objects.
- f. Improves with More Data and Training - Deep learning models generally become more accurate as they receive more data and undergo longer training.
- g. Supports Multiple Data Types - Can process various forms of data, including images, speech, text, video, and sensor data.
- h. Uses Backpropagation and Optimization - Models improve by adjusting neural network weights using algorithms such as backpropagation and gradient descent.
- i. High Accuracy in Complex Tasks - Achieves excellent performance in tasks such as image recognition, natural language processing, and speech recognition.
- j. Ability to Perform End-to-End Learning - Learns directly from raw input data to produce final outputs without requiring multiple separate processing steps.

Types of Deep Learning	
Artificial Neural Networks (ANN)	An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of the human brain. It consists of interconnected artificial neurons that process information and learn patterns from data. ANNs are the foundation of many machine learning and deep learning applications. An ANN is mainly composed of three types of layers: a. Input Layer - Receives the input data from the external environment. b. Hidden layer - Located between the input and output layers that perform complex computations and feature extraction. c. Output Layer - Produces the final result or prediction. Applications: Stock prediction, Medical diagnosis, Customer behavior prediction. Working Procedure of ANN 1. Input gathering - The network receives input data through the input layer. 2. Weight Assignment - Each connection between neurons has a weight that represents the importance of information being transferred. 3. Processing and Activation - Neurons calculate a weighted sum of inputs and apply an activation function to determine the output. 4. Forward Propagation - Information moves from the input layer through hidden layers to the output layer to generate a prediction. 5. Error Calculation - The predicted output is compared with the actual output to calculate the error. 6. Backpropagation and Learning - The network adjusts weights to reduce errors and improve accuracy.
Convolutional Neural Networks (CNN)	Convolutional Neural Network (CNN) is a deep learning model mainly designed for processing images and visual data. Applications: Face recognition, Object detection, Medical image analysis, Autonomous vehicles
Recurrent Neural Networks	Recurrent Neural Network (RNN) is a deep learning model designed for sequential data where previous information influences future predictions. Applications: Speech recognition, Language translation, Text generation, Stock

(RNN)	market analysis
Generative Adversarial Networks (GANs)	GANs are deep learning models that generate new data similar to real data using two competing networks: a generator and a discriminator. Applications: Image generation, AI art creation, Video generation, Data augmentation
Transformer Networks	Transformers are deep learning architectures that use attention mechanisms to understand relationships between elements in data, especially text. Applications: Large language models (LLMs), Machine translation, Text generation, Question-answering systems
Long Short Term Memory Networks (LSTM)	LSTM is an advanced type of RNN designed to overcome the problem of forgetting important information in long sequences. Applications: Chatbots, Machine translation, Voice assistants, Time-series forecasting
Autoencoders	Autoencoders are neural networks that learn to compress data into a smaller representation and reconstruct it back. Applications: Image compression, Noise removal, Fraud detection, Anomaly detection

Aspect	Traditional Machine Learning	Deep Machine Learning
Feature Engineering	Manual feature extraction required	Automatically learns features
Data Requirement	Works with smaller datasets	Requires large datasets
Model Complexity	Simpler models (linear, tree-based)	Complex multi-layer neural networks
Computation	Low computational cost	High computational cost (GPUs/TPUs)
Training Time	Faster training	Slower, time-consuming training
Performance	Good for structured data	Excellent for large-scale, unstructured and complex data
Unstructured Data	Limited capability	Excels in images, audio, and text
Interpretability	Easier to interpret	Hard to interpret (black-box nature)
Scalability (large data)	Limited scalability	Highly scalability
Flexibility	Less flexible for complex patterns	Highly flexible for complex patterns
Hardware Dependency	Works on standard CPUs	Often requires specialized hardware
End-to-End Learning	Requires separate preprocessing steps	Supports end-to-end learning

	How do neural networks distinguish useful features from non-useful features in Computer Vision?
--	---

In Computer Vision, neural networks distinguish useful features from non-useful features through a process called automatic feature learning. Instead of manually selecting features, deep learning models (especially Convolutional Neural Networks - CNNs) learn important patterns directly from images during training.

Process of Feature Learning

1. **Input Image Processing** - The neural network receives an image as input in the form of numerical values (pixel intensities). Each pixel contains information about color, brightness, and texture.
2. **Feature Extraction Using Layers** - Neural networks contain multiple layers that learn different levels of features:
 - a. **Low Level Features** - Detect simple patterns such as - Edges, Lines, Corners, Color changes, Textures. Example: In a face image, the first layers may detect edges around the eyes, nose, and mouth.
 - b. **Mid Level Features** - Combine simple features to identify more complex patterns such as - Shapes, Object parts, Patterns. Example: A network may identify an eye, ear, or wheel.
 - c. **High Level Features** - Recognize complete objects and meaningful concepts such as - Face, Car, Animal, Human body

How the Network differentiate Useful Features from non-useful features -

- a. **Weight Adjustment During Training** - Each connection in a neural network has a weight that represents the importance of a feature. During training, the network adjusts these weights using backpropagation. Features that help produce correct predictions receive higher importance.
- b. **Error-Based Learning** - The network compares its prediction with the actual answer. If a feature improves accuracy, its weight is strengthened. If a feature causes errors, its importance is reduced.
- c. **Convolution Operation (CNNs)** - CNNs use filters (kernels) that scan images to detect important patterns. Different filters learn different features such as edges, textures, and shapes. Useful features create stronger activation signals.
- d. **Activation Functions** - Activation functions decide whether a neuron should respond to a particular feature. Important patterns produce stronger activations, while irrelevant patterns produce weak or no activation.
- e. **Pooling and Feature Selection** - Pooling layers reduce unnecessary details while keeping important information. They help the network focus on dominant features and ignore noise.

Feed Forward Neural Networks

Feed-forward Neural Network (FNN) is a type of artificial neural network in which information flows in a single direction i.e. from the input layer through hidden layers to the output layer without loops or feedback. It is mainly used for pattern recognition tasks like image and speech classification.

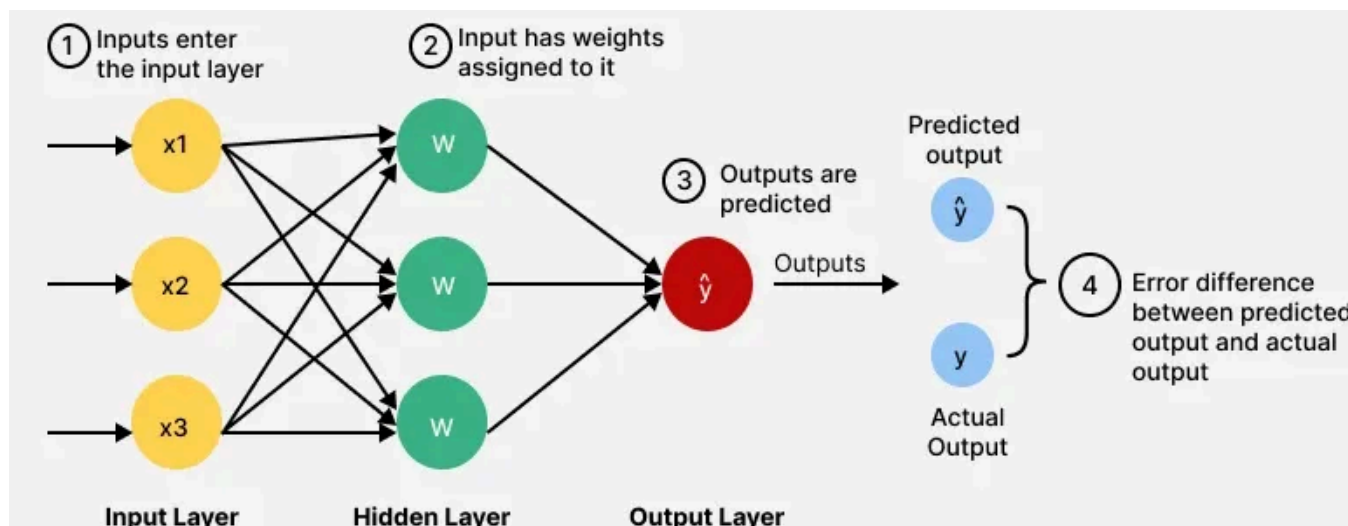
Structure of a Feed-forward Neural Network –

Feed-forward Neural Networks have a structured layered design where data flows sequentially through each layer.

- a. **Input Layer**: The input layer consists of neurons that receive the input data. Each neuron in the input layer represents a feature of the input data.
- b. **Hidden Layers**: One or more hidden layers are placed between the input and output layers. These layers are responsible for learning the complex patterns in the data. Each neuron in a hidden layer applies a weighted sum of inputs followed by a non-linear activation function.
- c. **Output Layer**: The output layer provides the final output of the network. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.
- d. **Activation Functions** - Activation functions introduce non-linearity into the network enabling it to learn and model complex data patterns. Common activation functions include:

$$\begin{aligned}
 \text{Sigmoid: } \sigma(x) &= \frac{1}{1+e^{-x}} \\
 \text{Tanh: } \tanh(x) &= \frac{e^x - e^{-x}}{e^x + e^{-x}} \\
 \text{ReLU: } \text{ReLU}(x) &= \max(0, x)
 \end{aligned}$$

Each connection between neurons in these layers has an associated weight that is adjusted during the training process to minimize the error in predictions.



Training a Feed-forward Neural Network – Training a Feed-forward Neural Network involves adjusting the weights of the neurons to minimize the error between the predicted output and the actual output. This process is typically performed using back-propagation and gradient descent.

- Forward Propagation:** During forward propagation the input data passes through the network and the output is calculated.
- Loss Calculation:** The loss (or error) is calculated using a loss function such as Mean Squared Error (MSE) for regression tasks or Cross-Entropy Loss for classification tasks.
- Back-propagation:** In back-propagation the error is propagated back through the network to update the weights. The gradient of the loss function with respect to each weight is calculated and the weights are adjusted using gradient descent.

Note: During training, a feed-forward neural network performs forward pass followed by back-propagation to update weights, while during prediction only the forward pass is used.

Gradient Descent - Gradient Descent is an optimization algorithm used to minimize the loss function by iteratively updating the weights in the direction of the negative gradient. Common variants of gradient descent include:

- Batch Gradient Descent:** Updates weights after computing the gradient over the entire dataset.
- Stochastic Gradient Descent (SGD):** Updates weights for each training example individually.
- Mini-batch Gradient Descent:** It Updates weights after computing the gradient over a small batch of training examples.

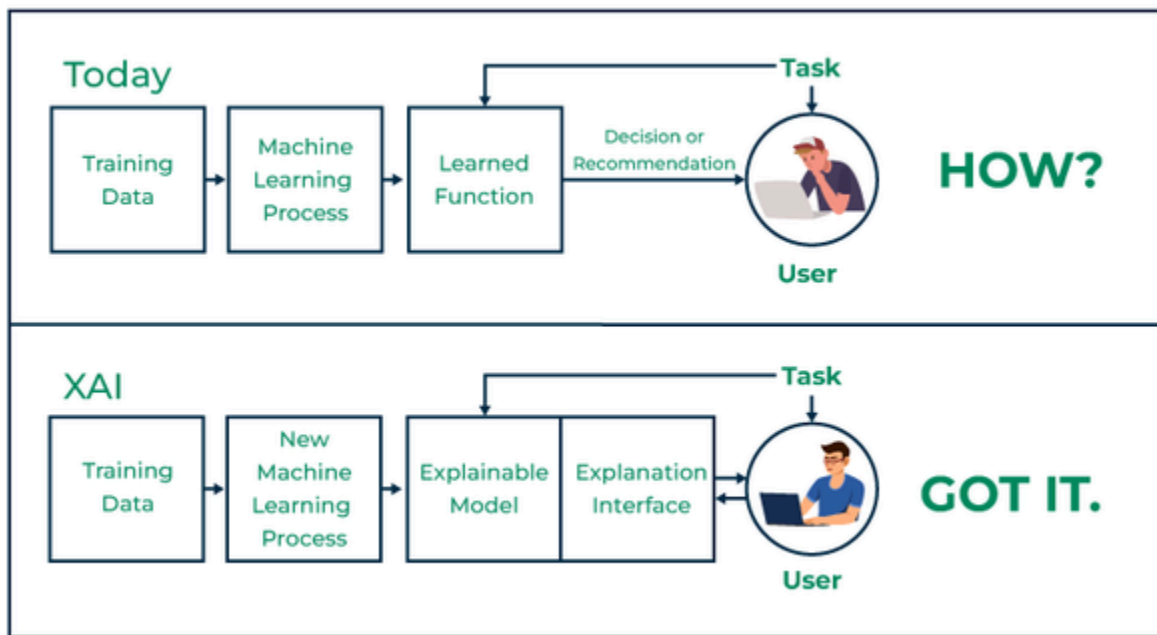
Evaluation of feed-forward neural network

- Evaluating the performance of the trained model involves several metrics:
- Accuracy:** The proportion of correctly classified instances out of the total instances.
- Precision:** The ratio of true positive predictions to the total predicted positives.
- Recall:** The ratio of true positive predictions to the actual positives.
- F1 Score:** The harmonic mean of precision and recall, providing a balance between the two.
- Confusion Matrix:** A table used to describe the performance of a classification model, showing the true positives, true negatives, false positives and false negatives.

What is Explainable AI (XAI)? How does it work? Explain with examples [47th BCS (CSE)]

Explainable artificial intelligence (XAI) refers to a collection of procedures and techniques that enable machine learning algorithms to produce output and results that are understandable and reliable for human users.

In other words, Explainable artificial intelligence (XAI) as the word represents is a process and a set of methods that helps users by explaining the results and output given by AI/ML algorithms.

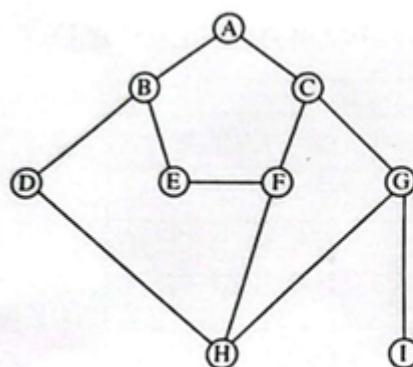


How does Explainable AI work?

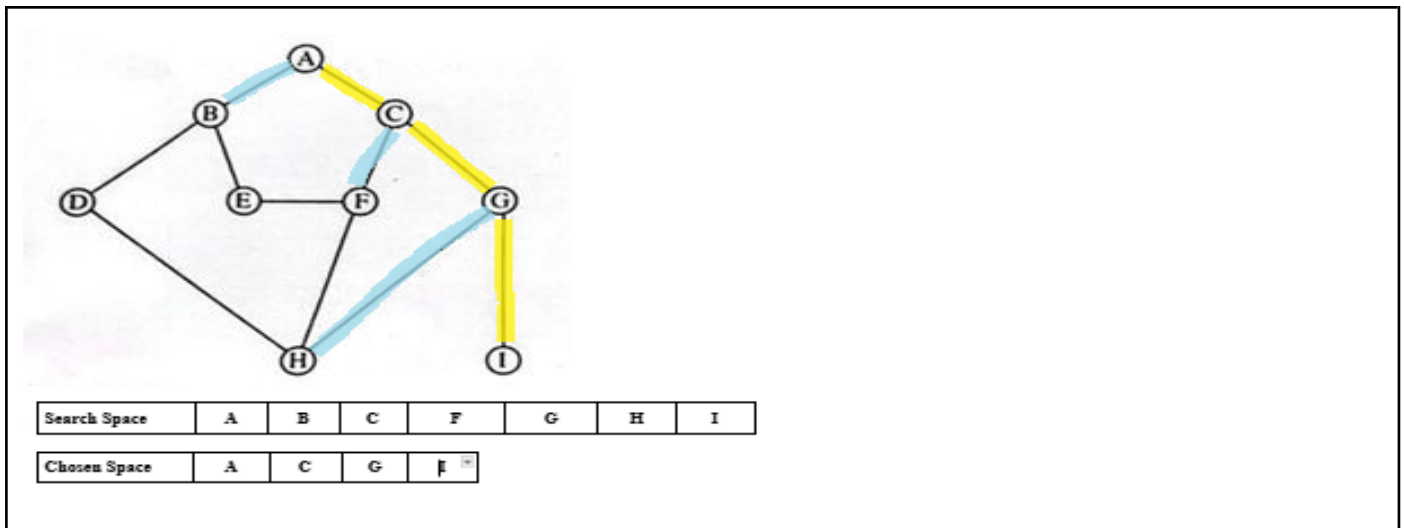
However, in general, explainable AI architecture can be thought of as a combination of three key components:

1. **Machine learning model**:- The machine learning model is the core component of explainable AI, and represents the underlying algorithms and techniques that are used to make predictions and inferences from data. This component can be based on a wide range of machine learning techniques, such as supervised, unsupervised, or reinforcement learning, and can be used in a range of applications, such as medical imaging, natural language processing, and computer vision.
2. **Explanation algorithm**:- The explanation algorithm is the component of explainable AI that is used to provide insights and information about the factors that are most relevant and influential in the model's predictions. This component can be based on different explainable AI approaches, such as feature importance, attribution, and visualization, and can provide valuable insights into the workings of the machine learning model.
3. **Interface**:- The interface is the component of explainable AI that is used to present the insights and information generated by the explanation algorithm to humans. This component can be based on a wide range of technologies and platforms, such as web applications, mobile apps, and visualizations, and can provide a user-friendly and intuitive way to access and interact with the insights and information generated by the explainable AI system.

Consider the following tree and heuristic value $h(n)$ at each node of the tree. if A is the start node and I is the destination node, explain how the best first search algorithm can be used to find the destination node. Also show the path to the destination



Heuristic Value	
Node	$h(n)$
A	12
B	9
C	4
D	8
E	6
F	7
G	3
H	3
I	0



Explain the difference between Deterministic Finite Automata (DFA) and Non Deterministic Finite Automata (NFA).

A **Deterministic Finite Automaton (DFA)** is a finite state machine in which, for every state and every input symbol, there is exactly one next state.

A **Non-Deterministic Finite Automaton (NFA)** is a finite state machine in which, for a state and an input symbol, there may be - multiple next states or one next state or no next state.

Feature	DFA	NFA
Full Form	Deterministic Finite Automata	Non-Deterministic Finite Automata
Transition	Exactly one transition for each input symbol from every state	May have multiple transitions for the same input symbol
Determinism	Deterministic	Non-deterministic
ϵ -transitions	Not allowed	Allowed (in ϵ -NFA)
Number of Next States	Only one next state	One, many, or none
Transition Function	$\delta: Q \times \Sigma \rightarrow Q$	$\delta: Q \times \Sigma \rightarrow 2^Q$
Computation Path	Single path	Multiple possible paths
Acceptance Condition	Accepts if the single computation path ends in final state	Accepts if at least one path ends in final state
Complexity	Easier to implement	Easier to design
Speed of Simulation	Faster	Slower due to branching

Prove that for every NFA, there exists an equivalent DFA that recognizes the same language.

Given NFA,

- States: $Q = \{q_0, q_1\}$
- Input alphabet: $\Sigma = \{0, 1\}$
- Start state: q_0
- Final states: $F = \{q_1\}$

- Transition function δ :
 - $\delta(q_0, 0) = \{q_0, q_1\}$
 - $\delta(q_0, 1) = \{q_0\}$
 - $\delta(q_1, 1) = \{q_1\}$
 - For undefined transitions, assume empty set ($\emptyset$)

Step 1: Construct states of the DFA	Each state of the DFA corresponds to a subset of Q. The power set of Q is: $P(Q) = \{\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}$ These will be the states of the DFA.																									
Step 2: Define the start state of the DFA	The start state of the DFA is the subset containing the NFA start state: $S_0 = \{q_0\}$																									
Step 3: Define the transition function for the DFA	For each DFA state (subset of NFA states) and input symbol, the transition is the union of the NFA transitions from each state in the subset. - $\delta_D(\emptyset, 0) = \emptyset$ - $\delta_D(\emptyset, 1) = \emptyset$ - $\delta_D(\{q_0\}, 0) = \delta(q_0, 0) = \{q_0, q_1\}$ - $\delta_D(\{q_0\}, 1) = \delta(q_0, 1) = \{q_0\}$ - $\delta_D(\{q_1\}, 0) = \delta(q_1, 0) = \emptyset$ (no transition defined) - $\delta_D(\{q_1\}, 1) = \delta(q_1, 1) = \{q_1\}$ - $\delta_D(\{q_0, q_1\}, 0) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$ - $\delta_D(\{q_0, q_1\}, 1) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_1\} = \{q_0, q_1\}$																									
Step 4: Define the final states of the DFA	Any DFA state (subset) that contains at least one NFA final state is a final state of the DFA. - $\emptyset$ contains no final states $\rightarrow$ not final - $\{q_0\}$ contains no final states $\rightarrow$ not final - $\{q_1\}$ contains $q_1 \rightarrow$ final - $\{q_0, q_1\}$ contains $q_1 \rightarrow$ final																									
Step 5: Summary of the DFA	<table><tr><th>DFA State</th><th>Corresponding NFA States</th><th>On input 0</th><th>On input 1</th><th>Final State?</th></tr><tr><td>$S_0 = \{q_0\}$</td><td>q_0</td><td>$\{q_0, q_1\}$</td><td>$\{q_0\}$</td><td>No</td></tr><tr><td>$S_1 = \{q_0, q_1\}$</td><td>q_0, q_1</td><td>$\{q_0, q_1\}$</td><td>$\{q_0, q_1\}$</td><td>Yes</td></tr><tr><td>$S_2 = \{q_1\}$</td><td>q_1</td><td>$\emptyset$</td><td>$\{q_1\}$</td><td>Yes</td></tr><tr><td>$S_3 = \emptyset$</td><td>None</td><td>$\emptyset$</td><td>$\emptyset$</td><td>No</td></tr></table>	DFA State	Corresponding NFA States	On input 0	On input 1	Final State?	$S_0 = \{q_0\}$	q_0	$\{q_0, q_1\}$	$\{q_0\}$	No	$S_1 = \{q_0, q_1\}$	q_0, q_1	$\{q_0, q_1\}$	$\{q_0, q_1\}$	Yes	$S_2 = \{q_1\}$	q_1	$\emptyset$	$\{q_1\}$	Yes	$S_3 = \emptyset$	None	$\emptyset$	$\emptyset$	No
DFA State	Corresponding NFA States	On input 0	On input 1	Final State?																						
$S_0 = \{q_0\}$	q_0	$\{q_0, q_1\}$	$\{q_0\}$	No																						
$S_1 = \{q_0, q_1\}$	q_0, q_1	$\{q_0, q_1\}$	$\{q_0, q_1\}$	Yes																						
$S_2 = \{q_1\}$	q_1	$\emptyset$	$\{q_1\}$	Yes																						
$S_3 = \emptyset$	None	$\emptyset$	$\emptyset$	No																						

	Final notes: - The start state of the DFA is $S_0 = \{q_0\}$. - The final states of the DFA are $S_1 = \{q_0, q_1\}$ and $S_2 = \{q_1\}$. - The DFA transitions are fully deterministic. Thus, the subset construction method produces a DFA equivalent to the given NFA.
--	---

	Discuss different phases of a compiler
--	---

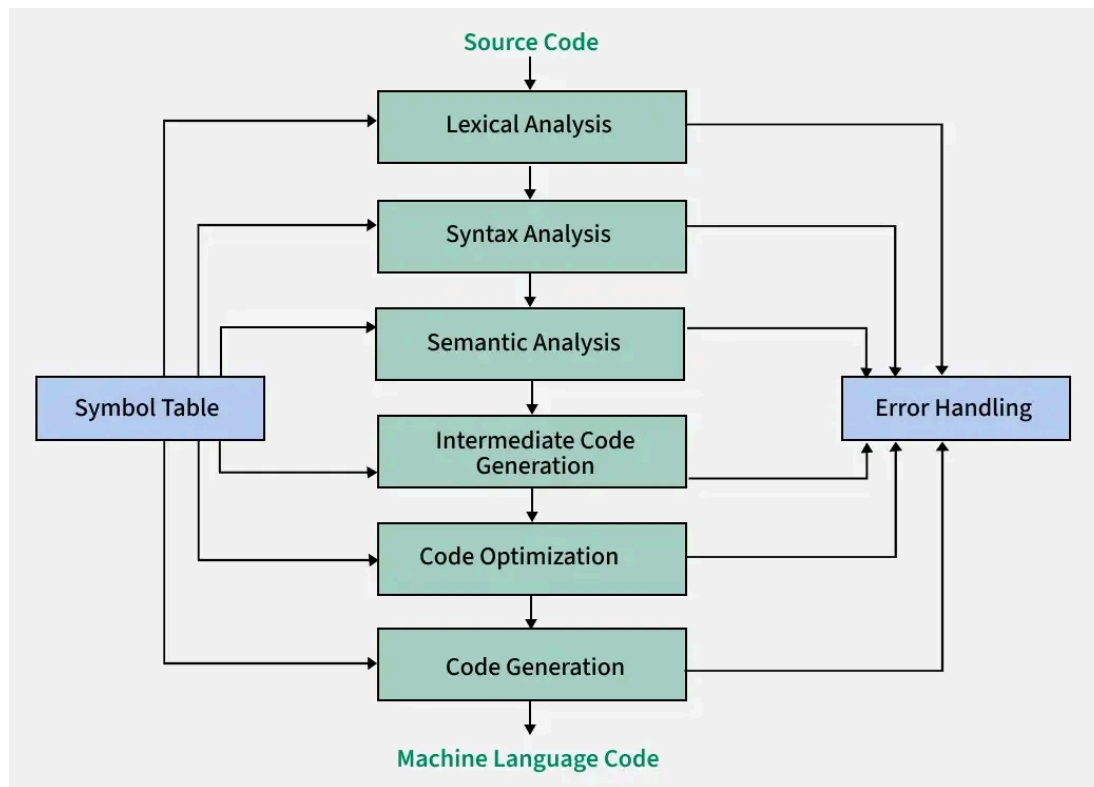
A compiler converts a high-level programming language into machine language through several phases. Each phase performs a specific task in the compilation process.

1	Lexical Analysis	This is the first phase of a compiler. The lexical analyzer scans the code character by character and groups them into tokens based on language rules. A token is the smallest meaningful unit (keywords, identifiers, constants, operators, punctuation). It simplifies input for the next phases of compilation.
2	Syntax Analysis	Phase of the compiler that ensures the source code follows the grammatical rules of the programming language. It verifies that the sequence of tokens produced by the lexical analyzer is arranged correctly according to the language's syntax. If the code is not structured according to the language rules, the syntax analyzer generates appropriate error messages. To represent the structure of the source code, syntax analysis uses parse trees or syntax trees. - Parse Tree: A parse tree is a tree-like structure that represents the syntactic structure of the source code. It shows how the tokens relate to each other according to the grammar rules. Each branch in the tree represents a production rule of the language, and the leaves represent the tokens. - Syntax Tree: A syntax tree is a more abstract version of the parse tree. It represents the hierarchical structure of the source code but with less detail, focusing on the essential syntactic structure. It helps in understanding how different parts of the code relate to each other.
3	Semantic Analysis	Semantic analysis is the phase of the compiler that ensures the source code is logically meaningful. While syntax analysis checks the structure of the program, semantic analysis checks its meaning. This phase identifies semantic errors such as type mismatches, undeclared variables, and invalid operations. Some key checks performed during semantic analysis include: - Type Checking: The compiler ensures that operations are performed on compatible data types. For example, trying to add a string and an integer would be flagged as an error because they are incompatible types. - Variable Declaration: It checks whether variables are declared before they are used. For example, using a variable that has not been defined earlier in the code would result in a semantic

		error.
4	Intermediate Code Generation	Intermediate Code Generation is the phase of the compiler in which the source code is converted into an intermediate representation (IR). This intermediate code lies between the high-level source code and the final machine code. Importance of Intermediate Code: 1. Platform Independence: Since intermediate code is machine-independent, the same intermediate representation can be reused, but code generation must be performed again for each target platform. 2. Simplifying Optimization: Intermediate code provides a structured and simplified representation of the program, making it easier to apply optimization techniques such as: a. Dead Code Elimination: Removing parts of the code that don't affect the program's output. b. Loop Optimization: Improving loops to make them run faster or consume less memory. c. Common Subexpression Elimination: Reusing previously calculated values to avoid redundant calculations. 3. Easier Translation: Intermediate code is closer to machine code but not tied to a specific machine. This makes the final code generation process smoother and more efficient. This step is handled in the back end of the compile
5	Code Optimization	Code Optimization is the process of improving the intermediate or target code to make the program run faster, use less memory, or be more efficient, without altering its functionality. Common Techniques: 1. Constant Folding: Precomputing constant expressions. 2. Dead Code Elimination: Removing unreachable or unused code. 3. Loop Optimization: Improving loop performance through invariant code motion or unrolling. 4. Strength Reduction: Replacing expensive operations with simpler ones.
6	Code Generation	Code Generation is the final phase of a compiler, where the intermediate representation of the source program (e.g., three-address code or abstract syntax tree) is translated into machine code or assembly code. The source code written in a higher-level language is transformed into a lower-level language that results in a lower-level object code, which should have the following minimum properties: ● It should carry the exact meaning of the source code. ● It should be efficient in terms of CPU usage and memory management.

Phase	Input	Output	Primary Tables Managed
Lexical Analysis	Source Code	Token Stream	Symbol Table, Literal Table
Syntax Analysis	Token Stream	Parse Tree / Syntax Tree	Parse Table, Symbol Table
Semantic Analysis	Syntax Tree	Annotated Syntax Tree	Symbol Table (Type checking)

Intermediate Code Generation	Annotated Syntax Tree	Intermediate Code	Temporary Variable Table
Code Optimization	Intermediate Code	Optimized Intermediate Code	Flow Graphs, Loop Tables
Code Generation	Optimized Intermediate Code	Target Code (Assembly/Machine)	Register & Address Descriptors



	Difference between Compiler and Interpreter
--	--

Basis	Compiler	Interpreter
Definition	Translates the entire program into machine code at once	Translates and executes one line at a time
Execution Speed	Faster execution after compilation	Slower execution
Error Detection	Displays all errors after compilation	Displays errors one by one during execution
Output	Generates object/executable file	Does not generate object code
Translation Method	Whole program is translated together	Program is translated line by line
Memory Usage	Requires more memory	Requires less memory
Debugging	More difficult	Easier
Runtime Requirement	Compiled program can run without compiler	Interpreter is needed every time

Program Execution	Execution starts after entire compilation	Execution starts immediately line by line
Efficiency	More efficient for large programs	Less efficient for large programs
Examples	C, C++, Go	Python, JavaScript, Ruby